Matching People to Jobs

The theory, the code, and the report behind Delta Code 2026, Problem Statement 1

Notes for a curious reader (no prior graph theory needed)

October 2026

Contents

1 The problem: who gets which job? 2

1.1 The tempting shortcut: "first come, first served" . . . 2

2 Graphs and matchings 3

2.1 Roles with several seats . . . 3

3 Augmenting paths and Berge's Lemma 4

4 Flow networks: a plumbing view 5

4.1 The residual graph: "what can still change?" . . . 5

5 Doing it fast: Hopcroft–Karp (Dinic's idea) 6

5.1 How many steps does it take? . . . 6

6 The same problem as a table of equations (linear algebra view) 7

7 Reading the code: `allocate.py` 7

7.1 Making the data . . . 7

7.2 The greedy baseline . . . 8

7.3 Building the pipe network . . . 8

7.4 Starting from greedy (warm start) . . . 9

7.5 Step 1 of each phase: BFS levels . . . 9

7.6 Step 2 of each phase: DFS blocking flow . . . 9

7.7 The main loop and reading the answer . . . 10
7.8 Checks and benchmarks . . . 10
7.9 The tests . . . 10
7.10 How to run . . . 10

8 Reading the outputs **10**

8.1 `assignment.csv` . . . 10
8.2 `matching_vis.png` . . . 11
8.3 `report.md` . . . 11

9 Glossary and a quick self-check **12**

In plain words

This document explains, from scratch, everything behind the project: *why* a quick-and-easy way of handing out jobs leaves people unplaced, *what* mathematics fixes it, *how* the program `allocate.py` carries that mathematics out, and *how to read* the three outputs (`assignment.csv`, `matching_vis.png`, `report.md`). Gold boxes like this one restate the idea in everyday language.

1 The problem: who gets which job?

A company has **2,400 candidates** and **320 roles**. Each role has a limited number of seats (between 3 and 10), and each candidate is only eligible for a few roles (1 to 5). We want to place *as many candidates as possible*, with nobody in a role they are not eligible for and no role over its seat limit.

This is the same puzzle as residency placements, assigning jobs to servers in a data centre, or seating guests so everyone gets a table they are happy with.

1.1 The tempting shortcut: "first come, first served"

The obvious method, called **greedy**, goes through the candidates one by one and puts each into the first eligible role that still has a seat. It is fast and easy to explain. But every decision is final, and that is its weakness.

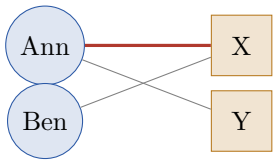
Example 1.1: The smallest trap

Two roles, each with one seat: X and Y. Two candidates:

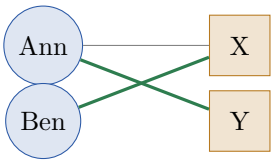
- **Ann** is eligible for X and Y (she lists X first).

- **Ben** is eligible for X only.

Greedy handles Ann first and gives her X. Now Ben arrives: X is full and he cannot do Y, so **Ben is left out**. Total placed: 1. But if Ann had taken Y, *both* would have a job. Total possible: 2.



Greedy: 1 placed (Ben stranded)



Optimal: 2 placed

Figure 1: Thick lines are the chosen placements. Ann moving from X to Y "makes room" for Ben.

Key idea

Greedy fails because it never *reconsiders*. The cure is a systematic way of saying: "if someone stuck needs a seat, can we shift a chain of already-placed people, each one step, so a seat opens up?" That chain is called an **augmenting path**, and it is the star of this project.

On the project's data, greedy places **1,709** candidates while the best possible is **1,901**. The difference, **192 people**, is the *Greedy Capacity Gap* that the problem statement asks us to measure.

2 Graphs and matchings

Definition 2.1: Graph, bipartite graph

A **graph** is a set of dots (*vertices*) joined by lines (*edges*). It is **bipartite** if the dots split into two teams so that every line joins a dot of one team to a dot of the other. Here the teams are U (candidates) and V (roles); a line u–v means "candidate u is eligible for role v".

Definition 2.2: Matching

A **matching** M is a set of edges in which no vertex is used twice. In words: every candidate gets at most one role and (in the simplest case) every role at most one candidate. Its size $|M|$ is the number of edges. A **maximum matching** is one with the largest possible size.

2.1 Roles with several seats

Real roles have capacity c_v (the number of seats). The standard trick is to **split a role into c_v identical slots**: a role with 3 seats becomes three one-seat roles, each connected to the same

candidates. A capacity-respecting assignment is then just an ordinary matching in the bigger graph, so all matching theory applies unchanged. (The program does not literally make copies; it uses capacities directly, as explained in Section 4.)

In plain words

A matching is a legal set of placements. "Maximum" means: no legal set places more people.

3 Augmenting paths and Berge's Lemma

Fix a matching M (say, the one greedy produced). Call a vertex **free** if M does not use it (an unplaced candidate, or an empty seat).

Definition 3.1: Augmenting path

An **M-augmenting path** is a path that

1. starts at a free candidate and ends at a free seat, and
2. alternates: a line *not* in M, then a line in M, then not in M, …, ending with a line not in M.

In the trap example the path is: *Ben* $\to X$ (not in M) $\to$ *Ann* (this is the line Ann–X, in M) $\to Y$ (not in M, and Y is a free seat). Reading it as a story: "Ben takes X, Ann leaves X and takes Y."

Flipping the path means: lines that were in M go out, lines that were out come in. Every person on the path ends up placed (the two ends gain a partner, the middle people just swap), so the matching grows by exactly one:

$$|M_{\text{new}}| = |M| + 1.$$

Theorem 3.2: Berge's Lemma

A matching M is maximum **if and only if** there is no M-augmenting path.

In plain words

If a "shift-everyone-one-step" fix exists, you are not done. If no such fix exists *anywhere*, you have the best possible answer. So the recipe is: keep finding fixes until none are left.

Proof. ($\Rightarrow$) If an augmenting path exists, flipping it gives a legal matching one larger, so M was not maximum.

($\Leftarrow$) Suppose M is not maximum; let M' be a bigger matching. Draw only the lines that are in exactly one of M and M'; call this picture H. Each dot touches at most one line of M and at most one of M', so in H every dot touches at most 2 lines. Hence H falls apart into separate paths and loops, and along each, lines alternate between M and M'. Loops and even-length paths have equally many M and M' lines. Since $|M'| > |M|$, some piece has *more* M' lines; it must be

an odd-length path that begins and ends with M' lines. Take an end dot x. Its single line in H is an M' line. If M also touched x by some line e, then $e \notin M'$ (as M' uses only one line at x), so e would lie in H and give x two lines in H, a contradiction. So both ends are free for M, and the piece is an M-augmenting path. □

4 Flow networks: a plumbing view

Finding augmenting paths cleverly is easiest in the language of *flows*, which you can picture as water in pipes.

Definition 4.1: The network G'

Add two new dots: a **source** s and a **sink** t. Add pipes

- $s \to u$ with capacity 1, for every candidate u (each person is placed at most once);
- $u \to v$ with capacity 1, for every eligible pair;
- $v \to t$ with capacity c_v, for every role v (its number of seats).

Pushing one unit of water from s to t along $s \to u \to v \to t$ means "place candidate u in role v". The **maximum flow** (the most water you can push) equals the **maximum number of hires**.

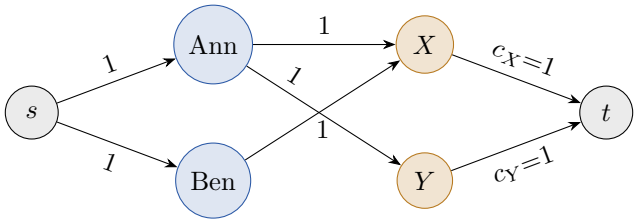


Figure 2: The trap example as a network. The most water that can flow from s to t is 2.

4.1 The residual graph: "what can still change?"

After some water is flowing, draw the **residual graph**: a pipe with spare room can still carry more (forward), and a pipe carrying water can have its water *pushed back* (backward). A path from s to t in this residual picture is exactly an augmenting path. A backward step $X \to$ Ann reads "Ann gives up X".

Theorem 4.2: Max-flow = min-cut

The largest possible flow equals the smallest total capacity of any set of pipes whose removal disconnects s from t (a *cut*).

In plain words

There is always a bottleneck. If the program ends with no route left from s to t, the set of dots still reachable from s marks the bottleneck, and its capacity is exactly the number of hires. That is a second, independent proof of optimality. On our data both numbers are **1,901**.

5 Doing it fast: Hopcroft–Karp (Dinic's idea)

Finding one augmenting path at a time works but can be slow. Hopcroft and Karp's improvement is to find *many shortest fixes at once*. Each **phase** has two steps:

1. **BFS (levels).** Starting at s, label every dot by how many steps away it is, using only pipes with spare room (like ripples spreading on a pond). If t never gets a label, there are no fixes left: **stop, we are optimal**.
2. **DFS (blocking flow).** Using only pipes that go from level k to level $k{+}1$, repeatedly walk from s towards t, push one unit along each path found, and update. Dead ends are remembered and never tried again in this phase. Continue until no such walk exists.

Then repeat. After each phase, the length of the shortest remaining fix strictly grows.

In plain words

First fix everyone who can be fixed with a short shuffle, then those needing longer shuffles, and so on. In our run the shuffle lengths were 5, 7, 9, 11 steps, and only **four phases** were needed.

5.1 How many steps does it take?

Let N be the number of dots in the network and $|E|$ the number of pipes.

One phase costs about $|E|$. The BFS touches each pipe once. In the DFS each pipe is a dead end at most once, and each pipe (except the last ones into t) has capacity 1, so it carries at most one path per phase.

There are only about $\sqrt{N}$ phases. After $\sqrt{N}$ phases every remaining fix is at least $\sqrt{N}$ steps long. Each person can be used by only one fix at a time (they have a single unit of throughput), so the leftover fixes are *disjoint*, and each uses at least $\sqrt{N}/2$ people. At most $N/(\sqrt{N}/2) = 2\sqrt{N}$ of them fit, and every phase handles at least one. So at most about $\sqrt{N}$ more phases.

$$\text{total time} \approx \underbrace{O(\sqrt{N})}_{\text{phases}} \times \underbrace{O(|E|)}_{\text{per phase}} = O(|E|\sqrt{|V|}).$$

Remark

The *guarantee* is generous; reality is kinder. For 2,400 candidates $\sqrt{N} \approx 52$ phases would be the worst case, but only 4 were needed because the greedy start was already close to optimal.

6 The same problem as a table of equations (linear algebra view)

The title of this problem says "Linear Algebra", so here is that view. Give every eligibility line $e = (u, v)$ a variable x_e that is 1 if we use it and 0 otherwise. Then:

$$\begin{aligned} \text{maximise} \quad & \sum_{e} x_e \quad \text{(total hires)} \\ \text{subject to} \quad & \sum_{e \ni u} x_e \le 1 \quad \text{for each candidate } u \quad \text{(at most one job)}, \\ & \sum_{e \ni v} x_e \le c_v \quad \text{for each role } v \quad \text{(seat limit)}, \\ & x_e \in \{0, 1\}. \end{aligned}$$

Written as $A\mathbf{x} \le \mathbf{b}$, the matrix A has one row per candidate and per role and one column per eligibility line; each column has exactly two ones (its candidate row and its role row). This is the **Integer Linear Program (ILP)**.

If we allow $0 \le x_e \le 1$ to be fractions, we get the **LP relaxation**, which is much easier to solve. In general fractions can cheat, but for bipartite matching the matrix A is **totally unimodular**: every square sub-table has determinant 0, +1 or −1, which forces the best LP solution to consist of whole numbers anyway. That is why, in the benchmark, LP, ILP and our flow method all report exactly 1,901.

In plain words

The "spreadsheet of constraints" version and the "water in pipes" version are two descriptions of the same problem. The spreadsheet solver is general-purpose; the pipes solver is specialised, so it is faster (about 3–4 times, in our tests).

7 Reading the code: `allocate.py`

The program is one file, built from small functions. Below, each piece is shown and explained in everyday terms.

7.1 Making the data

```
def generate(seed=2026, n_cand=2400, n_roles=320):
    rng = random.Random(seed)
    caps = [rng.choice([3, 4, 5, 6, 7, 8, 10]) for _ in range(n_roles)]
    w = [1.0 / (i + 1) ** 0.7 for i in range(n_roles)] # popularity
    rng.shuffle(w)
    ...
```

No real data came with the problem, so the program invents some *reproducibly*: the same "seed" always gives the same data. Seat counts are random; some roles are made more popular than others (that is realistic, and it is exactly what hurts greedy). Each candidate gets 1–5 eligible roles, in order of preference. The result is saved as `data_edges.csv` and `data_roles.csv`. To use real data, run the program with `--edges` and `--roles` instead.

7.2 The greedy baseline

```
def greedy(cand_adj, caps):
    load = [0] * len(caps)
    match = [-1] * len(cand_adj)
    for u, nb in enumerate(cand_adj):
        for v in nb:
            if load[v] < caps[v]:
                match[u] = v
                load[v] += 1
                break
    return match
```

`load[v]` counts how many seats of role v are taken. For each candidate u (in order), look through their roles; take the first one with a free seat; the `break` means "decision made, never revisited". `-1` means "not placed".

7.3 Building the pipe network

```
def _add(self, a, b, c):
    e = len(self.to)
    self.to += [b, a]
    self.cap += [c, 0]
    self.adj[a].append(e)
    self.adj[b].append(e + 1)
    return e
```

Every pipe is stored *together with its reverse pipe*, in neighbouring slots e and $e{+}1$. The forward pipe starts with capacity c; the reverse starts with 0. Pushing one unit through a pipe means *lowering* its capacity by 1 and *raising* its partner's by 1; that is how "pushing water back" (undoing a placement) is allowed. The partner of pipe `e` is `e ^ 1` (flip the last bit of the number): a compact trick.

The network constructor then creates pipes $s \to$ candidates (capacity 1), candidate $\to$ role for each eligibility line (capacity 1), and role $\to t$ (capacity c_v), exactly as in Definition 4.1.

7.4 Starting from greedy (warm start)

`push_initial` pre-loads the greedy placements into the network, as if water were already flowing along them. This matters for the report: every fix the algorithm then finds is a genuine *repair of a greedy mistake*, which is what lets us print "how greedy misallocations were undone".

7.5 Step 1 of each phase: BFS levels

```
def bfs(self):
    level = [-1] * self.n
    level[self.s] = 0
    q = deque([self.s])
    while q:
        x = q.popleft()
        for e in self.adj[x]:
            y = self.to[e]
            if self.cap[e] > 0 and level[y] < 0:
                level[y] = level[x] + 1
                q.append(y)
    return level
```

`level` is the "ripple distance" from the source; `-1` means not reached yet. We only walk along pipes with room left (`cap[e] > 0`). If the sink ends with level `-1`, no fix exists.

7.6 Step 2 of each phase: DFS blocking flow

```
def blocking_flow(self, level, trace):
    it = [0] * self.n
    flow = 0
    s, t = self.s, self.t
    while True:
        path, x = [], s
        while x != t:
            moved = False
            while it[x] < len(self.adj[x]):
                e = self.adj[x][it[x]]
                y = self.to[e]
                if self.cap[e] > 0 and level[y] == level[x] + 1:
                    path.append(e); x = y; moved = True
                    break
                it[x] += 1
            if not moved:
                if x == s:
                    return flow
                level[x] = -1 # dead end: prune
                e = path.pop(); x = self.to[e ^ 1]
                it[x] += 1
        for e in path:
            self.cap[e] -= 1; self.cap[e ^ 1] += 1
        flow += 1
        trace.append([self.to[e ^ 1] for e in path] + [t])
```

This is a walk from s towards t that only steps "one level deeper". `it[x]` remembers which pipe at dot x we try next, so no pipe is tried twice. If we get stuck, we mark the dot as a dead end and step back. When we reach t, we flip the pipes along the path (`cap[e] -= 1; cap[e ^ 1] += 1`), count one more hire, and save the path in `trace`; these saved paths become the "chains" in the report. When even s is stuck, the phase is over.

7.7 The main loop and reading the answer

`run()` alternates BFS and blocking flow until BFS cannot reach t; that moment is the proof of optimality (Berge). `extract()` then reads off who ended up where: a candidate-to-role pipe that is full (capacity 0) means "this candidate is placed in this role".

7.8 Checks and benchmarks

- `verify_feasible`: confirms every placement is eligible and no role exceeds its seats.
- **Min-cut check**: finds everything still reachable from s and adds up the capacity of pipes leaving that set; the program asserts it equals the number of hires.
- **networkx**: only to compute the max-flow value independently, and to draw the picture. The matching itself uses none of it.
- **scipy**: `milp` solves the ILP of Section 6 and `linprog` the LP; both are timed against our method on four growing data sizes.

7.9 The tests

`test_allocate.py` builds 300 small random puzzles, solves each by our method (with and without the greedy warm start), and demands that the result is legal, equals networkx's answer, and leaves no augmenting path. It also re-checks the main data set (1,901 hires).

7.10 How to run

```
pip install -r requirements.txt
python test_allocate.py # prints: all tests passed
python allocate.py # writes all outputs
```

8 Reading the outputs

8.1 `assignment.csv`

One row per candidate (2,400 rows). Columns: `candidate_id`; `role_id` (empty if unplaced); `status` (`ASSIGNED` or `UNASSIGNED`); `greedy_role_id` (what greedy had given them); `changed_vs_greedy`

(1 if the optimal plan moved them, which happened to 251 people who switched roles plus the 192 newly placed people that greedy had left out, 443 rows in all). Of the 2,400 candidates, 1,901 are placed and 499 are not: at that point no further reshuffling can help, because every route from them to a free seat is blocked (this is exactly the bottleneck found by the min-cut).

8.2 `matching_vis.png`

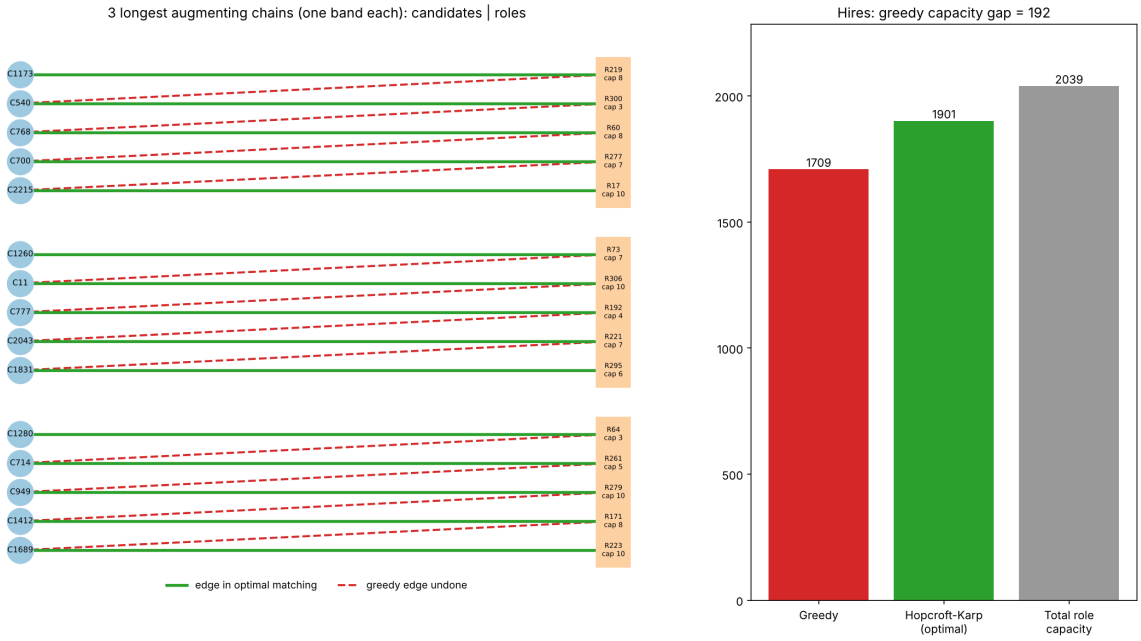


Figure 3: The project's figure. **Left:** three of the longest repair chains, one per horizontal band. Candidates on the left, roles on the right. A red dashed line is a placement greedy made that the algorithm *undid*; a green line is a placement in the final optimal plan. Each candidate's red line ends at the role that the next candidate down the chain takes: this staircase is the "everybody shifts one seat" story. **Right:** hires achieved by greedy (1,709), by the optimal method (1,901), and the total seats on offer (2,039).

Why not draw everything? With 2,400 candidates a full picture is an unreadable hairball, so the figure shows representative chains instead.

8.3 `report.md`

The report is the written argument. Its sections:

1. **Setup**: data sizes and the pipe-network model (Sections 1–5 here).
2. **Results**: greedy 1,709 versus optimal 1,901, so the gap is **192** (11.2% more than greedy). networkx, ILP and LP all agree.
3. **Berge's Lemma and optimality proof**: the proof from Section 3, plus the certificate that no fix remains and the cut equals 1,901.
4. **Complexity** $O(|E|\sqrt{|V|})$: the argument of Section 5.

5. **Path traces**: step-by-step stories of repair chains, e.g. "C1173 takes R219, C0540 vacates R219 and takes R300, …, C2215 takes R017 (a free seat)". Five placements and four removals give a net of *one extra person hired*, with nobody losing their job.
6. **Benchmark**: our method took about 25 ms; the ILP took about 123 ms. As the data doubles three times (to 9,600 candidates), our time grows roughly like $|E|^{1.27}$, well within the theory.
7. **Takeaways and caveats**: especially that the data is synthetic.
8. **Reproduce**: the commands.

Example 8.1: Reading one repair chain

"C1173 takes R219 → (C0540 vacates R219) → C0540 takes R300 → (C0768 vacates R300) → …". Phrases without parentheses are *new* placements; phrases in parentheses are people *leaving* a seat greedy gave them (backward pipes). Count the "takes": five placements; count the "vacates": four removals; net +1 hire.

9 Glossary and a quick self-check

Term	Meaning in this project
Greedy	Place each candidate in the first open eligible role; never reconsider.
Matching	A legal set of placements.
Free vertex	An unplaced candidate or an unfilled seat.
Augmenting path	A chain of shifts that places one more person.
Berge's Lemma	No augmenting path ⇔ the matching is the largest possible.
Flow / network	Water-in-pipes model; max flow = max hires.
Residual graph	Pipes showing what can still be added or undone.
BFS / DFS	Level labelling / walking one level deeper at a time.
Phase	One BFS plus one blocking-flow round.
Min-cut	The bottleneck; its capacity equals the maximum flow.
ILP / LP	Spreadsheet-style formulation (whole numbers / fractions allowed).
Greedy Capacity Gap	Optimal hires minus greedy hires (here 192).

Self-check questions

1. In the Ann–Ben example, write the augmenting path and say what flipping it does.
2. Why does greedy get stuck but the augmenting-path method does not?
3. What single fact tells the program it can stop?
4. Why is the LP answer a whole number here?
5. Why was the *warm start* from greedy useful for the report?

Answers. (1) Ben$\to$ X $\to$Ann$\to$ Y: Ben takes X, Ann moves to Y; one more person placed. (2) Greedy never undoes a choice; the method can reverse placements along a chain. (3) The BFS cannot reach the sink: no augmenting path remains (Berge). (4) The constraint table is totally unimodular. (5) Every fix found is then a repair of a specific greedy decision, which can be narrated.